

Essentials of Data Science Laboratory - 2304102L - 2026 - 2304102L

Dashboard Contents Calendar Assessments Syllabus

Search course ctrl + k

- 1. Practical 1
- 2. Practical 2
- 3. Practical 3
- 4. Practical 4
- 5. Practical 5



Pickup from where you left off!

Student Information
Practical 4 • Practice Lab Assignment

Resume

- Practical 1**
Unit • 100% completed
- Practical 2**
Unit • 100% completed
- Practical 3**
Unit • 100% completed

Activate Windows
Go to Settings to activate Windows.

Essentials of Data Science Laboratory - 2304102L - 2026

Search course ctrl + k

- 1. Practical 1
 - 1.1. Practice Lab Assignment
 - 1.1.1. Calculate Momentum**
 - 1.1.2. Conditional Calculation Based on the...
 - 1.1.3. Days between Two Dates
 - 1.1.4. Reverse a Number
 - 1.1.5. Multiplication Table
 - 1.2. Lab Assignment
- 2. Practical 2
- 3. Practical 3
- 4. Practical 4
- 5. Practical 5

1.1.1. Calculate Momentum 03:31

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum p is calculated using the formula:

$$p = m \times v$$

where:
 m is the mass of the object (in kilograms).
 v is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Sample Test Cases +

calculate...

```
1 x=float(input())
2 y=float(input())
3 p=x*y
4 print(f"{p:.2f}kgm/s")
```

Submit

1.1.2. Conditional Calculation Based on the Number of Digits 12:28

Write a Python program that accepts an integer n as input. Depending on the number of digits in n .

Constraints:
 $1 \leq n \leq 999$

Input Format:
• The input consists of a single integer n .

Output Format:
• If n is a single-digit number, print its square.
• If n is a two-digit number, print its square root (rounded to two decimal places).
• If n is a three-digit number, print its cube root (rounded to two decimal places).
• Else print "Invalid".

Sample Test Cases +

```
condition... Submit
1 x=int(input())
2 v if(x>0 and x<=9):
3   print(x**2)
4 v elif(x>9 and x<=99):
5   print(f"{x**(0.5):.2f}")
6 v elif(x>99 and x<=999):
7   print(f"{x**(1/3):.2f}")
8 v else:
9   print("Invalid")
10
```

Terminal Test cases

Activate Windows
Go to Settings to activate Windows.

1.1.3. Days between Two Dates

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format *YYYY – MM – DD*.
- The second line contains the second date in the format *YYYY – MM – DD*.

Output Format:

- An integer representing the number of days between the given dates.

Note:

- The first date should always be considered earlier than the second date.

Sample Test Cases

```
noOfDays...
1 from datetime import datetime
2
3 date1_str = input()
4 date2_str = input()
5
6 date1 = datetime.strptime(date1_str, '%Y-%m-%d')
7 date2 = datetime.strptime(date2_str, '%Y-%m-%d')
8
9 x = date2 - date1
10 days_diff = x.days
11
12 print(days_diff)
```

Activate Windows
Go to Settings to activate Windows.

Terminal Test cases

Debugger

1.1.4. Reverse a Number 08:16

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

Sample Test Cases +

reverseN... Submit

```
1 n=int(input())
2 r=0
3 while n!=0:
4     d=n%10
5     r=r*10+d
6     n=n//10
7
8 print(r)
```

Terminal Test cases

Activate Windows
Go to Settings to activate Windows.

1.1.5. Multiplication Table 03:50

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:

<number> x <multiplier> = <result>

Note:

- The character "x" is used in the output to represent multiplication.
- Refer to the visible test-cases for more clarity.

Sample Test Cases

```
1 n=int(input())
2 for i in range (1,11):
3     print(f"{n} x {i} = {n*i}")
4
5
6
```


1.2.1. Pass or Fail

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer n , the number of courses.
- The second input will be n integers representing the marks of the student in each of the n courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Note:

- Aggregate Percentage: Average performance across all courses, calculated by summing all

passorFa...

```
1 # Read input
2 num_courses = int(input())
3 marks = list(map(int, input().split()))
4
5 # Check fail condition
6 if any(mark < 40 for mark in marks):
7     print("Fail")
8 else:
9     aggregate = sum(marks) / num_courses
10
11 # Print exactly as required
12 print(f"Aggregate Percentage: {aggregate:.2f}")
13
14 if aggregate > 75:
15     print("Grade: Distinction")
16 elif aggregate >= 60:
17     print("Grade: First Division")
18 elif aggregate >= 50:
19     print("Grade: Second Division")
20 else:
21     print("Grade: Third Division")
```

Activate Windows
Go to Settings to activate Windows.

Terminal Test cases

Submit

1.2.2. Fibonacci Series 16:53

Write a Python program that uses recursion to print the first n terms of the Fibonacci series.

Input Format:

- A single integer n representing the number of terms to generate.

Output Format:

- A single line containing the first n terms of the Fibonacci sequence, separated by spaces.

Sample Test Cases +

fibonacci... Submit

```
1 def fibonacci(n):
2     if n == 1:
3         return 0
4     elif n == 2:
5         return 1
6     else: return fibonacci(n-1) + fibonacci(n-2)
7
8
9 n = int(input())
10 for i in range(1, n+1):
11     print(fibonacci(i), end=" ")
12
13
```


1.2.3. Pattern - 1 15:24

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Sample Test Cases

rightangl...

```
1 n=int(input())
2 for i in range(1,n+1):
3     for j in range(i):
4         print("*",end=" ")
5     print()
```

Terminal Test cases

Activate Windows
Go to Settings to activate Windows.

2.1.1. List operations 32:21

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:

1. Add
2. Remove
3. Display
4. Quit

The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:

- **Add:** Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
- **Remove:** Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
- **Display:** Displays the current list of integers. If the list is empty, display "List is empty".
- **Quit:** Exits the program.
- The program should handle invalid menu choices by displaying "Invalid choice". Ensure that the program continues to prompt the user until they choose to quit (option 4).

Sample Test Cases

```
listOps.py
1 def main():
2     integer_list = []
3
4     while True:
5         print("1. Add")
6         print("2. Remove")
7         print("3. Display")
8         print("4. Quit")
9
10        choice = input("Enter choice: ")
11
12        if choice == '1':
13            try:
14                val = int(input("Integer: "))
15                integer_list.append(val)
16                print(f"List after adding: {integer_list}")
17            except ValueError:
18                print("Invalid input")
19
20        elif choice == '2':
21            if not integer_list:
22                print("List is empty")
23            else:
24                try:
```

2.1.2. Dictionary Operations

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:

1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.

Input and Output Format:

1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

Note:

- All operations must be performed using dictionary methods.
- Perform deletion only if possible; leave the dictionary unchanged.
- Refer to the visible test cases for better understanding and strictly match with the input/output.

```
DictOper...
1 # Initial dictionary with 10 predefined records
2 student = {
3     1: "Amit",
4     2: "Riya",
5     3: "Kiran",
6     4: "Neha",
7     5: "Arjun",
8     6: "Pooja",
9     7: "Rahul",
10    8: "Sneha",
11    9: "Vikram",
12   10: "Anjali"
13 }
14
15 print("Original Dictionary:", student)
16
17 # Insertion
18 new_key = int(input())
19 new_value = input()
20 student[new_key] = new_value
21 print("After Insertion:", student)
22
23 # Update
24 upd_key = int(input())
```


2.2.2. Captain of the Team 00:34

You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

Input Format:
The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.

Output Format
The output should be the height (in centimeters) of the tallest player.

Sample Test Cases

```
captainof...
1 # Read 11 player heights
2 heights = list(map(int, input().split()))
3
4 # Print tallest player
5 print(max(heights))
```

2.2.1. Linear search Technique 00:26

Write a program to check whether the given element is present or not in the array of elements using linear search.

- Input format:**
- The first line of input contains the array of integers which are separated by space
 - The last line of input contains the key element to be searched

- Output format:**
- If the element is found, print the index.
 - If the element is not found, print **Not found**.

Sample Test Case:
Input:
1 2 3 4 3 5 6
3
Output:
2

Sample Test Cases

```
CTP1709... Submit
1 # Read array input
2 arr = list(map(int, input().split()))
3
4 # Read key
5 key = int(input())
6
7 # Linear search
8 found = False
9
10 for i in range(len(arr)):
11     if arr[i] == key:
12         print(i)
13         found = True
14         break
15
16 if not found:
17     print("Not found")
```

Terminal Test cases

Activate Windows
Go to Settings to activate Windows.

3.1.1. Numpy Array Operations 04:12

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using `ndim`
- The shape of the array using `shape`
- The total number of elements in the array using `size`

Assume all input elements are valid integers.

Input Format:

- The first line contains two space-separated integers, `r` and `c`, representing the number of rows and the number of columns.
- The next `r` lines contain `c` space-separated integers representing the elements of the array, entered row by row.

Output Format:

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

Note: Use `reshape()` function to reshape the input array with the specified number of rows and columns.

Sample Test Cases

```
numpyarr...
1 import numpy as np
2 import numpy as np
3
4 r, c = map(int, input().split())
5 elements = []
6
7 for _ in range(r):
8     elements.extend(list(map(int, input().split())))
9
10 arr = np.array(elements).reshape(r, c)
11
12 print(arr)
13 print(arr.ndim)
14 print(arr.shape)
15 print(arr.size)
16
```


5.2.11. Scatter Plot for Age vs. Fare by Survived

02:39

⌵

🔗

🔗

—

Write a Python code to plot a scatter plot showing the relationship between the 'Age' and 'Fare' columns in the Titanic dataset, with points color-coded by survival status. The scatter plot should display the following specifications:

1. Use the **Age** column for the x-axis and the **Fare** column for the y-axis.
2. Color the points based on the **Survived** column: **Red** for passengers who did not survive (**Survived = 0**). **Blue** for passengers who survived (**Survived = 1**).
3. Set the title of the plot to **"Age vs. Fare by Survival"**.
4. Label the x-axis as **'Age'** and the y-axis as **'Fare'**.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
```

AgeFareS...

Submit

Debugger

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # Data Cleaning
8 data['Age'].fillna(data['Age'].median(), inplace=True)
9 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
12 # Convert categorical features to numeric
13 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
15
16 # Write your code here for Scatter Plot for Age vs. Fare by Survived
17 colors = data['Survived'].map({0: 'red', 1: 'blue'})
18 plt.scatter(data['Age'], data['Fare'], c=colors)
19 plt.title('Age vs. Fare by Survival')
20 plt.xlabel('Age')
21 plt.ylabel('Fare')
22 plt.show()
```

Terminal Test cases

Activate Windows
Go to Settings to activate Windows.

5.2.7. Box plot for Age Distribution

Write a Python code to plot a boxplot that shows the distribution of the 'Age' column from the Titanic dataset across different passenger classes. The boxplot should display the following specifications:

1. Use the **Pclass** column to group the data for the boxplot.
2. Set the title of the plot to **"Age by Pclass"**.
3. Remove the default subtitle with **plt.suptitle("")**.
4. Label the x-axis as **'Pclass'** and the y-axis as **'Age'**.

The Titanic dataset contains columns as shown below.

Pas sen ger Id	Sur vive d	Pcl ass	Na me	Sex	Age	Sib Sp	Par ch	Tick et	Far e	Cab in	Em bar ked

Sample Data:

```
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
```

BoxPlotF...

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # Data Cleaning
8 data['Age'].fillna(data['Age'].median(), inplace=True)
9 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
12 # Convert categorical features to numeric
13 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
15
16 # Write your code here for Box Plot for Age by Pclass
17 # Box Plot for Age by Pclass
18
19 # Create boxplot grouped by Pclass
20 data.boxplot(column='Age', by='Pclass')
21
22 # Add title and labels
23 plt.title("Age by Pclass")
24 plt.xlabel("Pclass")
```

Terminal Test cases

3.2.7. Student Data Analysis and Operations 02:14

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- **Print all student details:** Display the complete details of all students, including roll numbers and marks for all subjects.
- **Find total students:** Determine the total number of students in the dataset.
- **Print all student roll numbers:** Extract and print the roll numbers of all students.
- **Print Subject 1 marks:** Extract and print the marks of all students in Subject 1.
- **Find minimum marks in Subject 2:** Identify the lowest marks in Subject 2.
- **Find maximum marks in Subject 3:** Identify the highest marks in Subject 3.
- **Print all subject marks:** Display the marks of all students for each subject.
- **Find total marks of students:** Compute the total marks for each student across all subjects.
- **Find the average marks of each student:** Compute the average marks for each student.
- **Find average marks of each subject:** Compute the average marks for all students in each subject.
- **Find average marks of Subject 1 and Subject 2:** Compute the average marks for Subject 1 and Subject 2.
- **Find average marks of Subject 1 and Subject 3:** Compute the average marks for Subject 1 and Subject 3.
- **Find the roll number of the student with maximum marks in Subject 3:** Identify the student with the highest marks in Subject 3 and print their roll number.
- **Find the roll number of the student with minimum marks in Subject 2:** Identify the student with the lowest marks in Subject 2 and print their roll number.
- **Find the roll number of students who scored 24 marks in Subject 2:** Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.

Operation... Submit

```
1 import numpy as np
2
3 a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)
4
5
6 # 1. Print all student details
7 print("All student Details:\n",a)
8
9 # 2. print total students
10
11 print("Total Students:", len(a))
12
13 # 3. Print all student Roll numbers
14 print("All Student Roll Nos", a[:,0] --)
15
16 # 4. Print subject 1 marks
17 print("Subject 1 Marks", a[:,1])
18
19 # 5. print minimum marks of Subject 2
20 print("Min marks in Subject 2", a[:,2].min() ----)
21
22 # 6. print maximum marks of Subject 3
23 print("Max marks in Subject 3", a[:,3].max() ----)
24
```

Terminal Test cases

Activate Windows
Go to Settings to activate Windows.

3.2.6. Numpy: Searching, Sorting, Counting, Broadcasting

The given code in the editor takes a single array, array1, as space-separated integers as input from the user.

Additionally, it takes the following inputs:

- search_value: The value to search for in the array.
- count_value: The value to count its occurrences in the array.
- broadcast_value: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

1. **Searching:** Find the indices where search_value appears in array1 and print these indices.
2. **Counting:** Count how many times count_value appears in array1 and print the count.
3. **Broadcasting:** Add broadcast_value to each element of array1 using broadcasting, and print the resulting array.
4. **Sorting:** Sort array1 in ascending order and print the sorted array.

Input Format:

1. A single line containing space-separated integers representing array1.
2. An integer search_value represents the value to search for in the array.
3. An integer count_value represents the value to count in the array.
4. An integer broadcast_value represents the value to add to each element of the array.

Output Format:

1. The indices where search_value occurs in array1.
2. The count of occurrences of count_value in array1.
3. The array after adding the broadcast_value to each element.

```
1 import numpy as np
2
3 # Input array from the user
4 array1 = np.array(list(map(int, input().split())))
5
6 # Searching
7 search_value = int(input("Value to search: "))
8 count_value = int(input("Value to count: "))
9 broadcast_value = int(input("Value to add: "))
10
11
12 # 1. Searching (indices)
13 search_result = np.where(array1 == search_value)
14 print(search_result[0])
15
16 # 2. Counting occurrences
17 count_result = np.count_nonzero(array1 == count_value)
18 print(count_result)
19
20 # 3. Broadcasting addition
21 broadcast_result = array1 + broadcast_value
22 print(broadcast_result)
23
24 # 4. Sorting
```

4.2.5. Titanic Dataset Analysis and Data Cleaning

You are provided with the Titanic dataset containing Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

Pas sen ger id	Sur vive d	Pcl ass	Na me	Sex	Age	Sib Sp	Par ch	Tick et	Far e	Cab in	Em bar ked

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7
8 # 1. First 5 rows
9 print(data.head())
10
11 # 2. Last 5 rows
12 print(data.tail())
13
14 # 3. Shape
15 print(data.shape)
16
17 # 4. Info (print manually captured format)
18 print(data.info())
19
20 # 5. Describe
21 print(data.describe())
22
23 # 6. Missing values
24 print(data.isnull().sum())
```


3.2.5. Numpy: Copying and Viewing Arrays

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the `original_array` and assigning it to `view_array`.
- Creating a copy of the `original_array` and assigning it to `copy_array`.

After completing these steps, observe how modifying the view affects the `original_array`, while modifying the copy does not.

Input Format:

- A single line of space-separated integers.

Output Format:

- After modifying the view:

Original array after modifying view: <original_array>
View array: <view_array>

- After modifying the copy:

Original array after modifying copy: <original_array>
Copy array: <copy_array>

Sample Test Cases

```
1 import numpy as np
2
3 inputlist = list(map(int, input().split(" ")))
4
5 # Original array
6 original_array = np.array(inputlist)
7
8 # Create a view
9 view_array = original_array.view()
10
11 # Create a copy
12 copy_array = original_array.copy()
13
14 # Modify the view
15 view_array[0] = 99
16 print("Original array after modifying view:", original_array)
17 print("View array:", view_array)
18
19 # Modify the copy
20 copy_array[1] = 88
21 print("Original array after modifying copy:", original_array)
22 print("Copy array:", copy_array)
```


4.2.3. City that Sold the Most Products

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by city and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 # write the code..
10
11
12
13 # Group by City and calculate total quantity sold
14 city_sales = df.groupby('City')['Quantity'].sum()
15
16 # Find the city with the highest total quantity sold
17 best_city = city_sales.idxmax()
18
19
20 # Display the result
21 print(f"City sold the most products: {best_city}")
22
```

Terminal Test cases

Activate Windows
Go to Settings to activate Windows.

< Prev Reset Submit Next >

4.1.3. Student Information

00:31

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Note:

Refer to the displayed test cases for better understanding.

Sample Test Cases

+

studentin...

Submit

```
1 import pandas as pd
2
3 # Read the text file into a DataFrame
4 file = input()
5 data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age",
6 "Grade"])
7
8 # write your code here..
9
10 print("First five rows:")
11 print(data.head())
12
13 # 2. Calculate average age (2 decimal places)
14 avg_age = round(data["Age"].mean(), 2)
15 print("Average age:", avg_age)
16
17 # 3. Filter students with grade up to 'B'
18 filtered = data[data["Grade"].isin(["A", "B"])]
19
20 print("Students with a grade up to B")
21 print(filtered)
22
```

Terminal Test cases

Activate Windows
Go to Settings to activate Windows.

< Prev Reset Submit Next >



4.2.1. Month with the Highest Total Sales 03:44

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases +

monthFor... Submit

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8 # Convert Date to datetime
9 df['Date'] = pd.to_datetime(df['Date'])
10
11 # Create Month column (YYYY-MM format)
12 df['Month'] = df['Date'].dt.to_period('M').astype(str)
13
14 # Calculate total sales per row
15 df['Sales'] = df['Quantity'] * df['Price']
16
17 # Group by Month and sum sales
18 monthly_sales = df.groupby('Month')['Sales'].sum()
19
20 # Find the month with the highest total sales
21 best_month = monthly_sales.idxmax()
22 highest_sales = monthly_sales.max()
23
24 print(f"Best month: {best_month}")
```

Terminal Test cases

5.2.10. Scatter Plot for Age vs. Fare

00:47

🔍 🌙 📄 🔗 —

Write a Python code to plot a scatter plot showing the relationship between the 'Age' and 'Fare' columns in the Titanic dataset. The scatter plot should display the following specifications:

1. Use the **Age** column for the x-axis and the **Fare** column for the y-axis.
2. Set the title of the plot to **"Age vs. Fare"**.
3. Label the x-axis as **'Age'** and the y-axis as **'Fare'**.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
```

AgeFareS...

Submit

Debugger

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # Data Cleaning
8 data['Age'].fillna(data['Age'].median(), inplace=True)
9 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
12 # Convert categorical features to numeric
13 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
15
16 # Write your code here for Box Plot for Fare by Pclass
17
18 # Scatter Plot for Age vs Fare
19
20 # Create scatter plot
21 plt.scatter(data['Age'], data['Fare'])
22
23 # Add title and labels
24 plt.title("Age vs. Fare")
```

Terminal Test cases

4.1.1. Pandas - series creation and manipulation 01:43

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the `groupby` and `mean()` operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

Sample Test Cases +

```
1 import pandas as pd
2
3 # Take inputs from the user to create a list of numbers
4 numbers = list(map(int, input().split()))
5
6 # Create a Pandas series from the list of numbers
7 s = pd.Series(numbers)
8 # Grouping by even and odd numbers and calculating the mean
9 grouped = s.groupby(s % 2 == 0).mean()
10
11 # Display the mean of even and odd numbers with labels
12 grouped.index = ['Even' if is_even else 'Odd' for is_even in
13 grouped.index]
14 print("Mean of even and odd numbers:")
15 print(grouped)
```


4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below.

Pas senger id	Surv ived	Pcl ass	Na me	Sex	Age	Sib Sp	Par ch	Tick et	Fare	Cab in	Em bar ked

Sample Data:

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data['FamilySize'] = data['SibSp'] + data['Parch']
7
8
9
10
11 # # 7. Get the number of passengers by gender
12
13
14
15 # # 8. Get the number of passengers by survival status
16
17
18
19 # # 9. Calculate the survival rate
20
21
22
23 # # 10. Calculate the survival rate by gender
24
```


4.2.4. Most Frequently Sold Product Pairs

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Explanation:

Transactions:

- 2025-01-01: Product A, Product B
- 2025-01-02: Product A, Product C
- 2025-01-03: Product B, Product A
- 2025-01-04: Product C, Product B

```
1 import pandas as pd
2 from itertools import combinations
3 from collections import Counter
4
5 # Prompt user to input the file name
6 file_name = input()
7
8 # Read data from the specified CSV file
9 df = pd.read_csv(file_name)
10
11
12
13 # Group products by Date
14 grouped = df.groupby('Date')['Product'].apply(list)
15
16 pair_counter = Counter()
17
18 # Generate product pairs for each date
19 for products in grouped:
20     # Remove duplicates within the same date if any
21     unique_products = sorted(set(products))
22
23     # Generate all combinations of 2 products
24     pairs = combinations(unique_products, 2)
```

Terminal Test cases

5.2.6. Bar Plot for Survival by Embarked

Write a Python code to plot a stacked bar chart showing the survival count for passengers based on their embarkation location in the Titanic dataset.

The chart should display the following specifications:

1. Use the **Embarked** column to determine the embarkation location. After converting this column into dummy variables (using **pd.get_dummies()**), plot the survival count based on the **Embarked_Q** column (representing passengers who embarked from Queenstown) in relation to survival.
2. Set the chart type to 'bar' and make it stacked.
3. Add the title "**Survival by Embarked**" to the chart.
4. Label the x-axis as '**Embarked**' and the y-axis as '**Count**'.
5. Include a legend to distinguish between survivors and non-survivors (label the legend as '**Survived**' and '**Not Survived**').

The Titanic dataset contains columns as shown below.

Pas sen ger id	Sur vive d	Pcl ass	Na me	Sex	Age	Sib Sp	Par ch	Tick et	Far e	Cab in	Em bar ked

Sample Data:

BarPlotOf...

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # Data Cleaning
8 data['Age'].fillna(data['Age'].median(), inplace=True)
9 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
12 # Convert categorical features to numeric
13 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
15
16 # Write your code here for Bar Plot for Survival by Embarked
17 survival_counts = data.groupby('Embarked_Q')
18 ['Survived'].value_counts().unstack().fillna(0)
19 survival_counts.plot(kind = 'bar', stacked = True)
20 plt.title('Survival by Embarked')
21 plt.xlabel('Embarked')
22 plt.ylabel('Count')
23 plt.legend(['Not Survived', 'Survived'])
24 plt.show()
```

Terminal Test cases

4.1.2. Dictionary to dataframe

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column.

```
35
36 import pandas as pd
37
38 # Provided dictionary of lists
39 data = {
40     'Name': ['Alice', 'Bob', 'Charlie'],
41     'Age': [25, 30, 35],
42 }
43
44 # Convert the dictionary to a DataFrame
45 df = pd.DataFrame(data)
46
47 # Display the original DataFrame
48 print("Original DataFrame:")
49 print(df)
50
51 # -----
52 # Add a new row
53 new_name = input("New name: ")
54 new_age = int(input("New age: "))
55 new_row = {'Name': new_name, 'Age': new_age}
56 df = pd.concat([df, pd.DataFrame([new_row])], ignore_index=True)
57 print("After adding a row:\n", df)
58
59 #
```